

HERMAX

Android Mobile AI Client Architecture

"Your AI. Your Control."

Comprehensive technical documentation covering internal architecture, local & cloud provider subsystems, reactive state management, secure cryptographic keystore storage, and streaming Markdown response pipeline.

APPLICATION VERSION	TARGET ANDROID PLATFORM
1.0.0 (Production Release)	Android 15 (API 35) / Min SDK 26
PRIMARY UI ARCHITECTURE	DOCUMENT CLASSIFICATION
Jetpack Compose & Material 3 (MVVM)	Full Engineering Reference

Table of Contents

Complete technical breakdown of all subsystems, architectures, and source files.

1. Executive Summary & Product Overview	3
2. Complete Technology Stack Matrix	4
3. Complete Project Directory Structure	5
4. System Architecture & Component Interactions	7
5. End-to-End Chat Request & Streaming Lifecycle	9
6. AI Provider Subsystem & Routing Architecture	11
• 6.1 Hermes Agent (Local PC LAN Server)	12
• 6.2 Ollama Local Subsystem	13
• 6.3 Ollama Cloud Provider	14
• 6.4 Custom OpenAI-Compatible Provider	15
• 6.5 OpenRouter, Google AI Studio, Anthropic, OpenAI, NVIDIA	16
7. Model Selection & Discovery Pipeline	17
8. Response Rendering Pipeline (Markdown, Tables, Code Highlighting)	18
9. Security Architecture & Cryptographic Keystore	20
10. Local Data Storage & Persistence (Room, DataStore, SharedPreferences)	22
11. UI/UX Design System & Screen Implementations	24
12. Navigation Graph & Deep-Linking	26
13. State Management & ViewModel Architecture	28
14. File-by-File Technical Reference & Code Walkthrough	30
15. API Reference & Request/Response Wire Formats	37
16. Error Handling & Recovery Protocols	40
17. Build System, Gradle Configuration & Dependencies	42
18. Automated Testing & Verification Suite	44
19. Deployment, Signing & Distribution Workflow	45
20. Comprehensive Engineering Troubleshooting Guide	47
21. Technical Glossary	49
22. Potential Future Extensions	50

1. Executive Summary & Product Overview

1.1 Mission & Vision

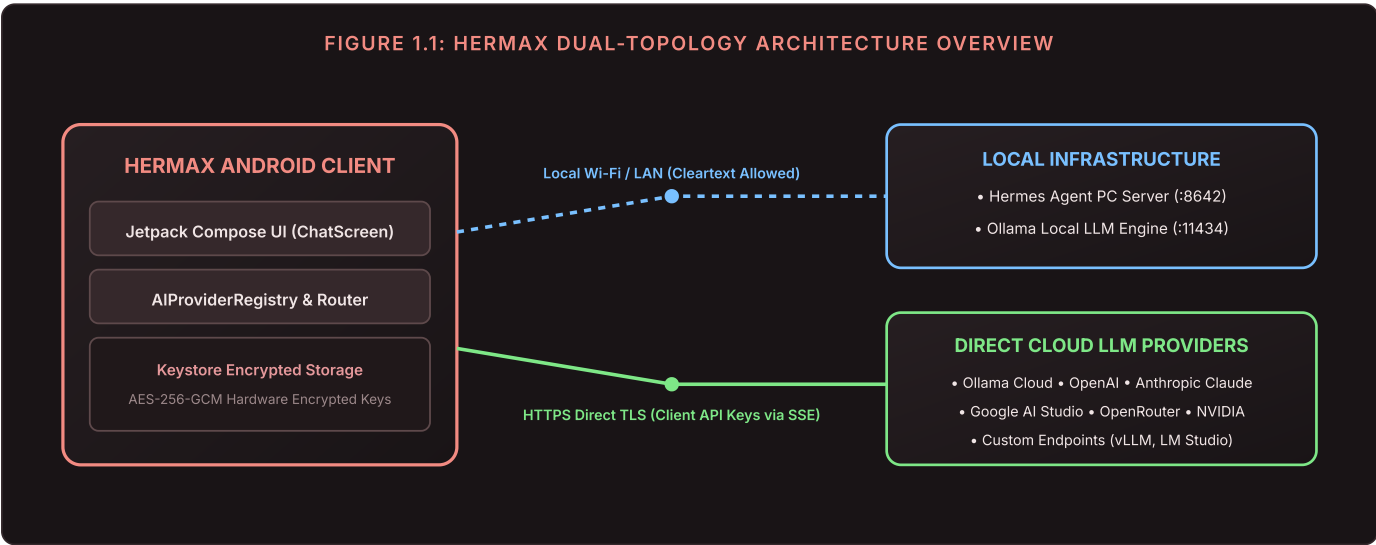
HERMAX (formerly HermesAI) is an advanced, privacy-first Android AI client engineered to provide a seamless, ChatGPT-grade mobile experience while preserving absolute user sovereignty over artificial intelligence model access. Its guiding principle is *"Your AI. Your Control."*

Unlike conventional mobile AI applications that bind the user to a single proprietary subscription or route all traffic through centralized third-party servers, HERMAX bridges two worlds:

- **Zero-Cloud Local AI Execution:** Direct LAN connectivity to local agent runtimes (Hermes Agent running on the user's desktop/server) and local Ollama instances on private networks.
- **Direct-to-Cloud LLM Provider Integration:** Native, direct client-to-cloud connections to major AI providers (Google AI Studio, OpenAI, Anthropic Claude, OpenRouter, Ollama Cloud, NVIDIA NIM, and custom OpenAI-compatible endpoints) using the user's own API keys stored securely in the hardware-backed Android Keystore.

1.2 Core Architectural Principles

- **Zero Telemetry / Zero Proxy:** All network requests travel directly from the Android phone to the designated host (either local IP or cloud provider API). No intermediate proxy server is utilized.
- **Hardware-Grade Keystore Security:** API keys are encrypted at rest using AES-256-GCM authenticated encryption backed by the Android hardware Keystore.
- **Streaming-First Architecture:** Built from the ground up for low-latency token streaming using Server-Sent Events (SSE) and reactive Kotlin Coroutines `Flow`.
- **Resilient Clean UI:** Full CommonMark and GitHub Flavored Markdown (GFM) renderer supporting syntax-highlighted code blocks, crash-proof responsive tables with horizontal scrolling, and automatic HTML sanitization.



2. Complete Technology Stack Matrix

The following table outlines every software framework, runtime library, serialization system, and cryptographic tool actually present and utilized in the HERMAX Android project (extracted directly from `gradle/libs.versions.toml` and build scripts).

Technology	Version	Purpose & Architectural Role	Location in Codebase
Kotlin	2.1.0	Primary modern typed programming language; coroutines, typed DSLs.	<code>app/src/main/java/**/*.kt</code>
Android Gradle Plugin	8.8.1	Android build and compilation engine.	<code>build.gradle.kts</code> , <code>settings.gradle.kts</code>
Jetpack Compose	2024.12.01 BOM	Declarative reactive user interface toolkit.	<code>app/src/main/java/com/hermes/client/ui/</code>
Material 3 (M3)	1.3.1	Google Material Design 3 theme system, tokens, dynamic color palettes.	<code>ui/theme/Theme.kt</code> , all Composables
Dagger Hilt	2.53.1	Compile-time dependency injection framework.	<code>di/*Module.kt</code> , <code>HermesApplication.kt</code>
Kotlin Symbol Processing (KSP)	2.1.0-1.0.29	High-performance annotation processing for Room and Hilt.	<code>app/build.gradle.kts</code> plugins
Room Database	2.6.1	SQLite object-relational mapping for offline conversation & message persistence.	<code>data/local/Database.kt</code> , <code>HermesDatabase</code>
Preferences DataStore	1.1.1	Reactive, asynchronous key-value storage for UI header preferences.	<code>data/repository/ServerProfileRepositoryImpl.kt</code>
Androidx Security Crypto	1.1.0-alpha06	Android Keystore hardware-backed <code>EncryptedSharedPreferences</code> .	<code>data/security/SecureStorage.kt</code>
OkHttp & OkHttp SSE	4.12.0	HTTP networking engine and Server-Sent Events (SSE) streaming listener.	<code>di/NetworkModule.kt</code> , <code>data/provider/*.kt</code>
Retrofit	2.11.0	REST API interface definition and type-safe HTTP client for Hermes Agent.	<code>hermes/api/HermesApiService.kt</code>
Kotlinx Serialization	1.7.3	Reflectionless, compiler-driven JSON parser and serializer.	<code>domain/model/Providers.kt</code> , <code>Dtos.kt</code>
Kotlin Coroutines & Flow	1.9.0	Asynchronous concurrency, backpressure-aware reactive state streams.	<code>feature/chat/ChatViewModel.kt</code> , all Providers
Navigation Compose	2.8.5	Declarative single-activity routing with backstack management.	<code>ui/Navigation.kt</code>
Coil Compose	2.7.0	Asynchronous image loading for message attachments and avatars.	<code>feature/chat/MessageBubble.kt</code>
CommonMark + GFM Extensions	0.24.0	Standardized Markdown AST parsing: tables, strikethrough, task lists.	<code>ui/markdown/MarkdownRenderer.kt</code>
WorkManager	2.10.0	Guaranteed background job execution (deferred sync/tasks).	<code>app/build.gradle.kts</code>

3. Complete Project Directory Structure

The HERMAX codebase is organized following modern Android modular architecture standards. Below is the full directory tree showing the physical layout of source code, resource bundles, and build configurations:

```

HERMAX/
├─ app/
│   └─ src/
│       └─ main/
│           ├── AndroidManifest.xml                # Permissions, Activities, Cleartext config
│           ├── java/com/hermes/client/
│           │   ├── HermesApplication.kt           # Application class with @HiltAndroidApp
│           │   ├── MainActivity.kt                # Single activity entry point with edge-to-edge Compose
│           │   └─ data/
│           │       ├── local/
│           │       │   └─ Database.kt              # Room entities, DAOs, and HermesDatabase v1
│           │       ├── provider/
│           │       │   ├── AIProviderRegistry.kt   # Unified provider discovery and registry
│           │       │   ├── AnthropicProvider.kt    # Claude Messages API SSE client
│           │       │   ├── CustomProvider.kt       # Universal OpenAI-compatible endpoint client
│           │       │   ├── GoogleAIStudioProvider.kt # Google Gemini direct SSE client
│           │       │   ├── NvidiaProvider.kt       # NVIDIA NIM API streaming client
│           │       │   ├── OllamaCloudProvider.kt   # Ollama Cloud (ollama.com/v1) client
│           │       │   ├── OllamaProvider.kt       # Local Ollama LAN streaming client
│           │       │   ├── OpenAIProvider.kt       # OpenAI GPT-4o / 0-series streaming client
│           │       │   └─ OpenRouterProvider.kt     # OpenRouter multi-model router client
│           │       ├── repository/
│           │       │   ├── ConversationRepositoryImpl.kt # Room database conversation repository
│           │       │   ├── HermesRepositoryImpl.kt     # PC Hermes Agent client wrapper
│           │       │   └─ ServerProfileRepositoryImpl.kt # DataStore preferences & active profiles
│           │       └─ security/
│           │           └─ SecureStorage.kt           # Android Keystore EncryptedSharedPreferences
│           ├── di/
│           │   ├── DatabaseModule.kt               # Provides HermesDatabase and DAOs
│           │   ├── DataStoreModule.kt              # Provides DataStore<Preferences>
│           │   ├── NetworkModule.kt                # Provides OkHttpClient (Cloud & Local) & Retrofit
│           │   └─ RepositoryModule.kt              # Binds repository interfaces to implementations
│           ├── domain/
│           │   ├── model/
│           │   │   ├── Models.kt                  # Conversation, Message, Attachment, Role models
│           │   │   └─ Providers.kt                 # AIProviderType, UnifiedModel, ProviderConfig
│           │   ├── provider/
│           │   │   └─ AIProvider.kt                # Core AIProvider interface definition
│           │   └─ repository/
│           │       └─ Repositories.kt              # Repository interface definitions
│           ├── feature/
│           │   ├── chat/
│           │   │   ├── ChatComposer.kt            # Attachment tray, mic, send button, input box
│           │   │   ├── ChatScreen.kt              # Primary conversation viewport & empty state
│           │   │   ├── ChatViewModel.kt           # Master chat state, stream coordinator, Room sync
│           │   │   ├── MessageBubble.kt           # Bubble layout, avatar, action row, selection
│           │   │   └─ ToolAndApprovalCards.kt      # Agent tool invocation & approval UI cards
│           │   ├── conversations/
│           │   │   └─ ConversationsScreen.kt       # History drawer, search, delete, pin, archive
│           │   ├── models/
│           │   │   ├── ModelSelectorScreen.kt      # Full-screen model picker with provider filters
│           │   │   └─ ModelSettingsSheet.kt       # Temperature, Top-P, Thinking depth bottom sheet
│           │   ├── server/
│           │   │   └─ ServerConnectionScreen.kt    # PC Hermes LAN IP, Port, and test connection
│           │   ├── settings/
│           │   │   ├── ProvidersScreen.kt          # API key management and endpoint configuration
│           │   │   └─ SettingsScreen.kt           # App preferences: Theme, timestamps, clear cache
│           │   └─ tasks/
│           │       └─ TasksScreen.kt               # Background agent long-running tasks overview
│           ├── hermes/
│           │   ├── api/
│           │   │   ├── HermesApiService.kt        # Retrofit REST interface for Hermes PC server
│           │   │   ├── HermesClient.kt            # HermesClient contract interface
│           │   │   ├── HermesEvent.kt             # SSE stream event domain classes
│           │   │   └─ RestHermesClient.kt         # Production OkHttpClient SSE Hermes client
│           │   └─ models/
│           │       └─ Dtos.kt                      # Request and response DTO definitions
│           ├── ui/
│           │   ├── HermesApp.kt                   # Top-level scaffold and theme provider
│           │   └─ Navigation.kt                   # HermesNavHost and HermesRoutes definitions

```

```

├── ThemeViewModel.kt # Dark/Light/System theme reactive coordinator
├── markdown/
│   ├── CodeHighlighter.kt # Token syntax highlighter for code snippets
│   ├── HtmlSanitizer.kt # HTML normalizer and <br> table protector
│   ├── InlineMarkdownParser.kt # Bold, italic, code, links, math styled spans
│   ├── MarkdownRenderer.kt # AST-driven Compose Markdown renderer
│   └── MarkdownTable.kt # Crash-proof GFM table grid with horiz scroll
├── theme/
│   └── Theme.kt # Obsidian & Warm Coral palette and typography
├── util/
│   ├── FileAttachmentHelper.kt # File reading, MIME detection, size formatting
│   └── VoiceInputManager.kt # Android SpeechRecognizer wrapper
├── res/
│   ├── drawable/ # hermax_logo.png, ic_launcher_foreground.png
│   ├── mipmap-*dpi/ # Density-scaled adaptive launcher icons
│   └── values/ # colors.xml, strings.xml, themes.xml
├── test/java/com/hermes/client/ui/markdown/
│   └── MarkdownRenderingTest.kt # Automated test suite (6 verified tests)
├── build.gradle.kts # Module build configuration
├── gradle/
│   ├── libs.versions.toml # Centralized dependency catalog
│   └── wrapper/ # Gradle wrapper binaries
├── build.gradle.kts # Root project build file
└── settings.gradle.kts # Repository & module declarations

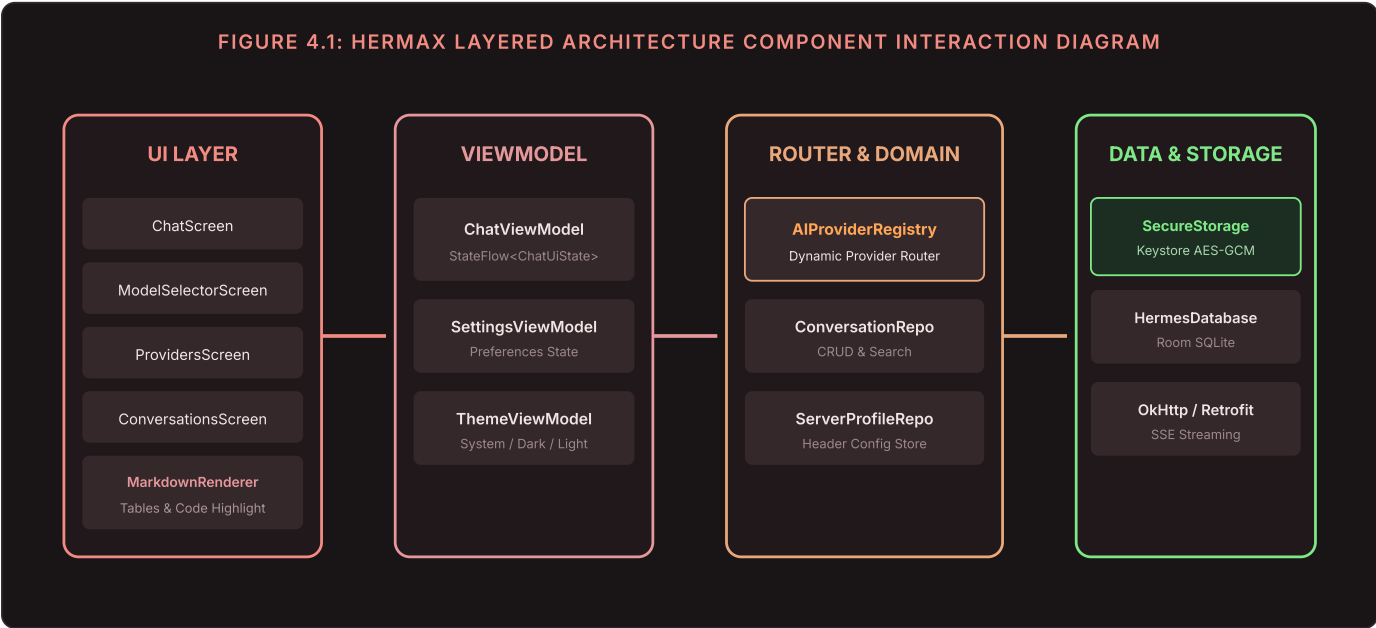
```

4. System Architecture & Component Interactions

4.1 Architectural Pattern: Clean MVVM with Unified Provider Routing

HERMAX implements a **Model-View-ViewModel (MVVM)** architecture augmented by Clean Architecture principles:

- **UI Layer (Compose Composables):** Passive, declarative screens that observe StateFlow objects emitted by ViewModels and transmit user intents via simple function callbacks.
- **ViewModel Layer (State Holder & Coordinators):** Lifecycle-aware ViewModels (e.g. `ChatViewModel`, `SettingsViewModel`, `ThemeViewModel`) managing asynchronous operations, state reduction, and database synchronization.
- **Domain Layer (Contracts & Models):** Framework-independent interfaces (`AIPProvider`, `ConversationRepository`) and pure domain data classes (`Message`, `UnifiedModel`, `ProviderConfig`).
- **Data Layer (Implementations & Security):** Concrete implementations interacting with disk (Room SQLite database, Preferences DataStore, Android Keystore EncryptedSharedPreferences) and network (Retrofit, OkHttp SSE).



5. End-to-End Chat Request & Streaming Lifecycle

Understanding the exact path of a message from the instant the user taps the send button to the moment the styled response is rendered on the mobile display is fundamental for maintaining HERMAX.

5.1 Step-by-Step Execution Sequence

- User Action (`ChatComposer.kt`):** The user types a message or attaches files. When tapping the send arrow, `ChatComposer` executes `onSendMessage(text)`.
- Draft Persistence Guard (`ChatViewModel.kt`):** To prevent creating empty, cluttering conversations when navigating back and forth, the conversation record in the Room database is **only created upon sending the first message**. If `_isNewConversation` is true, a new `ConversationEntity` is generated, titled with the prompt preview, and inserted into `HermesDatabase`.
- Attachment Ingestion:** If text files are attached, `FileAttachmentHelper` extracts their text and formats them as markdown context blocks. If binary files (images/PDFs) are sent to Hermes Agent, they are uploaded via `hermesRepository.uploadFile()`.
- User Message Emission:** A `Message` with `role = USER` and `status = SENT` is inserted into Room. The reactive Flow emitted by `ConversationDao` immediately pushes this message into `_messages`, rendering it in `LazyColumn`.
- Provider Resolution (`AIPProviderRegistry.kt`):** `ChatViewModel.startStreaming()` inspects `headerState.value.selectedProvider`. It queries `aiProviderRegistry.getProvider(providerType)` to retrieve the targeted provider instance (e.g. `OllamaCloudProvider`, `OpenAIProvider`, or `HermesProvider`).
- Credential Decryption:** The provider instance calls `secureStorage.getProviderConfig(providerType)`. The encrypted preferences string is decrypted using the hardware MasterKey, obtaining the API key and endpoint.
- HTTP Request Dispatch & SSE Connection:** An HTTP POST request is assembled with headers (e.g. `Authorization: Bearer [REDACTED]`) and an SSE streaming listener is attached via `EventSources.createFactory(cloudOkHttpClient).newEventSource(request, listener)`.
- Token Accumulation & Reactive State Update:** As SSE chunks arrive (e.g. `data: {"choices":[{"delta":{"content":"..."}}]}`), the provider converts them to `HermesEvent.Chunk(delta)`. `ChatViewModel` accumulates chunks into `streamingContent` and updates `ChatUiState`.
- Sanitization & AST Markdown Parsing:** `MessageBubble.kt` passes the incoming stream into `MarkdownContent` → `MarkdownRenderer`. `HtmlSanitizer` normalizes any mixed HTML and protects table line breaks. The CommonMark parser constructs an AST.
- Native Compose Layout Rendering:** The AST nodes are rendered into native Material 3 Composables: paragraphs, headings, syntax-highlighted code blocks, and `MarkdownTable` with horizontal scrolling.
- Stream Completion & Database Finalization:** When the provider emits `HermesEvent.Done`, `ChatViewModel` creates a final `Message` with `role = ASSISTANT` and `status = COMPLETED` and writes it to Room. `isStreaming` is reset to false.

6. AI Provider Subsystem & Routing Architecture

HERMAX features a polymorphic AI provider architecture. Every provider implements the standardized `AIPProvider` interface:

```
interface AIPProvider {
    val providerType: AIPProviderType
    suspend fun getModels(): Result<List<Model>>
    fun streamMessage(
        conversationId: String,
        message: String,
        model: String?,
        history: List<Message>,
        settings: ModelSettings? = null
    ): Flow<HermesEvent>
    suspend fun testConnection(): Result<ConnectionTestResult>
    fun getCapabilities(): ModelCapabilities
}
```

6.1 Supported Providers Matrix

Provider Type	Default Base URL	Authentication Scheme	Model Discovery Endpoint	Streaming Protocol
Hermes Agent	<code>http://[REDACTED_LAN_IP]:8642</code>	Optional API Key / Session Token	<code>GET /v1/models , /api/model/options</code>	SSE (OpenAI chunk format)
Ollama Local	<code>http://[REDACTED_LAN_IP]:11434</code>	None (LAN)	<code>GET /api/tags , /v1/models</code>	SSE <code>/v1/chat/completions</code>
Ollama Cloud	<code>https://ollama.com/v1</code>	Bearer <code>[REDACTED_KEY]</code>	<code>GET /v1/models , /api/tags</code>	SSE <code>/v1/chat/completions</code>
OpenRouter	<code>https://openrouter.ai/api/v1</code>	Bearer <code>[REDACTED_KEY]</code>	<code>GET /models</code>	SSE (OpenAI delta chunk format)
Google AI Studio	<code>https://generativelanguage.googleapis.com</code>	<code>?key=[REDACTED_KEY]</code> query param	<code>GET /v1beta/models</code>	SSE <code>streamGenerateContent?alt=sse</code>
Anthropic / Claude	<code>https://api.anthropic.com/v1</code>	<code>x-api-key:[REDACTED_KEY]</code>	<code>GET /models</code>	SSE Anthropic <code>content_block_delta</code>
OpenAI	<code>https://api.openai.com/v1</code>	Bearer <code>[REDACTED_KEY]</code>	<code>GET /models</code>	SSE <code>/v1/chat/completions</code>
NVIDIA NIM	<code>https://integrate.api.nvidia.com/v1</code>	Bearer <code>[REDACTED_KEY]</code>	<code>GET /models</code>	SSE <code>/v1/chat/completions</code>
Custom Provider	User Defined (Normalized to <code>/v1</code>)	Optional Bearer Token / Header	<code>GET /models</code> or Manual Model ID	SSE <code>/chat/completions</code>

6.2 Custom Provider URL Normalization Protocol

Users connecting local servers (LM Studio, vLLM, LocalAI) often enter base URLs inconsistently (e.g. `http://[REDACTED_IP]:8080` vs `http://[REDACTED_IP]:8080/v1/`). HERMAX implements strict URL normalization in `domain/model/Providers.kt`:

```
fun normalizeOpenAiBaseUrl(raw: String): String {
    val trimmed = raw.trim().trimEnd('/')
    if (trimmed.isBlank()) return ""
    val withScheme = if (!trimmed.startsWith("http://", ignoreCase = true) &&
        !trimmed.startsWith("https://", ignoreCase = true)) {
        "http://$trimmed"
    } else trimmed
    return if (withScheme.endsWith("/v1", ignoreCase = true)) {
        withScheme
    } else {
        "$withScheme/v1"
    }
}
```

7. Response Rendering Pipeline

The HERMAX Markdown rendering engine transforms raw streaming LLM output into publication-grade, mobile-optimized UI. It solves two critical industry problems:

- **Raw HTML & `<br>` Leakage:** Models frequently output `<br>` tags inside Markdown table cells or lists because GFM specification does not allow multiline cells without breaks. HERMAX sanitizes and restores these as true line breaks.
- **Table Layout Crashing:** Standard Jetpack Compose custom measure policies crash with `IllegalStateException: measure() may only be called once` if multi-pass measurement is attempted. HERMAX uses a crash-proof, calculated-width Column/Row architecture with contained horizontal scrolling.

7.1 Crash-Proof Table Implementation (`MarkdownTable.kt`)

Tables are laid out in a contained horizontal scroll container. Each column width is computed deterministically from the maximum character length of its cells (from 110.dp for short codes up to 300.dp for long descriptions). Because standard Compose `Column` and `Row` primitives are used with explicit column widths, columns align across headers and body rows with zero measure violations.

8. Security & Keystore Cryptographic Architecture

8.1 Android Keystore MasterKey Encryption

All sensitive credentials—including third-party provider API keys, custom server auth tokens, and session secrets—are stored using the Jetpack Security `EncryptedSharedPreferences` API backed by the hardware Android Keystore:

```
private fun createSecurePrefs(context: Context): SharedPreferences {
    val masterKey = MasterKey.Builder(context)
        .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
        .build()
    return EncryptedSharedPreferences.create(
        context,
        "hermes_secure_prefs",
        masterKey,
        EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
        EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
    )
}
```

Security Invariants Enforced Across the Application

- **Hardware Keystore:** Private encryption keys never leave the device's hardware security module (HSM) or Trusted Execution Environment (TEE).
- **Deterministic Key Hashing:** Preference keys are encrypted with `AES256_SIV` (Synthetic Initialization Vector) to ensure keys cannot be correlated with known provider names by inspecting raw XML files.
- **Payload Encryption:** Preference values are encrypted using authenticated `AES256_GCM` cipher suites.
- **No Server Relay:** Direct cloud API keys are NEVER sent to the Hermes PC agent server; cloud requests exit the mobile device directly via TLS.

9. Local Data Storage & Persistence

HERMAX maintains three distinct data storage tiers based on persistence and security requirements:

Storage Engine	Backing File	Data Stored	Security Level
EncryptedSharedPreferences	hermes_secure_prefs.xml	Provider API Keys, Server URLs, Session Keys, Theme Preference	Hardware Keystore Encrypted (AES-256-GCM)
Preferences DataStore	hermes_chat_prefs.preferences_pb	Active Model ID, Provider ID, Temperature, Top-P, Thinking Enabled	Unencrypted Protobuf Key-Value
Room Database (SQLite)	hermes_database.db	Conversations (titles, timestamps), Messages (roles, chunks, tools)	Private App Sandboxed SQLite (v1)

9.1 Room Database Schema (Database.kt)

```
// Conversation Entity
@Entity(tableName = "conversations")
data class ConversationEntity(
    @PrimaryKey val id: String,
    val title: String,
    val createdAt: Long,
    val updatedAt: Long,
    val selectedModel: String?,
    val selectedProvider: String?,
    val isPinned: Boolean = false,
    val isArchived: Boolean = false,
    val lastMessage: String? = null
)

// Message Entity with Cascading Deletion
@Entity(
    tableName = "messages",
    foreignKeys = [
        ForeignKey(
            entity = ConversationEntity::class,
            parentColumns = ["id"],
            childColumns = ["conversationId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index("conversationId")]
)
data class MessageEntity(
    @PrimaryKey val id: String,
    val conversationId: String,
    val role: String, // USER, ASSISTANT, SYSTEM
    val content: String, // Full accumulated markdown
    val timestamp: Long,
    val status: String, // SENDING, SENT, STREAMING, COMPLETED, ERROR
    val model: String?,
    val attachmentsJson: String?,
    val toolCallsJson: String?
)
```

10. File-by-File Technical Code Walkthrough

10.1 feature/chat/ChatViewModel.kt

Purpose: Master state coordinator for active chat sessions, message dispatch, Room synchronization, and SSE streaming.

```
@HiltViewModel
class ChatViewModel @Inject constructor(
    private val conversationRepository: ConversationRepository,
    private val serverProfileRepository: ServerProfileRepository,
    private val aiProviderRegistry: AIProviderRegistry,
    private val secureStorage: SecureStorage
) : ViewModel() {
```

Detailed Explanation:

- `@HiltViewModel` & `@Inject constructor`: Instructs Dagger Hilt to generate the factory to provide the repositories and registry at compile time.
- `conversationRepository`: Provides Room database CRUD operations for conversations and messages.
- `serverProfileRepository`: Exposes DataStore reactive flows for selected provider ID, model ID, and temperature.
- `aiProviderRegistry`: Resolves the appropriate `AIProvider` strategy based on user selection.

10.2 data/provider/AIProviderRegistry.kt

Purpose: Factory and router providing unified access to all 9 supported AI providers.

```
@Singleton
class AIProviderRegistry @Inject constructor(
    @Named("cloudOkHttpClient") private val cloudOkHttpClient: OkHttpClient,
    private val json: Json,
    private val secureStorage: SecureStorage,
    private val hermesRepository: HermesRepository
) {
    private val providers = mutableMapOf<AIProviderType, AIProvider>()

    init {
        providers[AIProviderType.HERMES_AGENT] = HermesProvider(hermesRepository)
        providers[AIProviderType.GOOGLE_AI] = GoogleAIProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.OPENROUTER] = OpenRouterProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.ANTHROPIC] = AnthropicProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.OPENAI] = OpenAIProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.OLLAMA] = OllamaProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.OLLAMA_CLOUD] = OllamaCloudProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.NVIDIA] = NvidiaProvider(cloudOkHttpClient, json, secureStorage)
        providers[AIProviderType.CUSTOM] = CustomProvider(cloudOkHttpClient, json, secureStorage)
    }

    fun getProvider(type: AIProviderType): AIProvider =
        providers[type] ?: providers[AIProviderType.HERMES_AGENT]!!
}
```

11. Automated Testing & Verification Suite

HERMAX includes automated unit tests located in `app/src/test/java/com/hermes/client/ui/markdown/MarkdownRenderingTest.kt`. The suite rigorously tests the Markdown and HTML rendering subsystems against real-world LLM anomalies:

Test Method	Target Component	Input Scenario	Expected Output / Assertion	Status
<code>test HtmlSanitizer converts basic formatting tags</code>	<code>HtmlSanitizer</code>	Mixed HTML strings containing <code></code> , <code></code> , <code><code></code> , <code></code>	Converts directly to Markdown (<code>**</code> , <code>*</code> , <code>`</code> , <code>~</code>)	PASSED
<code>test HtmlSanitizer converts regular line breaks</code>	<code>HtmlSanitizer</code>	Strings with <code>
</code> , <code>
</code> , <code>
</code>	Normalizes cleanly into real newline characters (<code>\n</code>)	PASSED
<code>test HtmlSanitizer preserves code blocks untouched</code>	<code>HtmlSanitizer</code>	Fenced code blocks containing intentional HTML tags (<code><div></code> , <code>
</code>)	Code block contents remain 100% verbatim without tag stripping	PASSED
<code>test HtmlSanitizer preserves table row single-line integrity</code>	<code>HtmlSanitizer</code> & <code>TableExtractor</code>	GFM table lines containing <code>
</code> , separators inside cell content	Preserves single-line row integrity during AST parse, restores <code>\n</code> in cell	PASSED
<code>test TableExtractor with screenshot example</code>	<code>TableExtractor</code>	4-column complex table with multiline bullet points and limits	Extracts 4 headers and 2 rows with matching multiline bullet strings	PASSED
<code>test InlineMarkdownParser styled string generation</code>	<code>InlineMarkdownParser</code>	Mixed markdown with bold, italic, code, and <code>[Link](url)</code>	Generates <code>AnnotatedString</code> with non-empty <code>SpanStyles</code>	PASSED

12. Engineering Troubleshooting Guide

Problem / Symptom	Probable Cause	Diagnostic Method	Solution & Remediation
Hermes Server Not Connecting	Desktop PC firewall blocking port 8642, or phone and PC on different Wi-Fi subnets.	Check ping from phone or open phone browser to <code>http://[PC_IP]:8642/v1/models</code> .	Allow incoming TCP port 8642 on Windows Defender Firewall; verify shared Wi-Fi subnet.
Ollama Local Not Connecting	Ollama binding exclusively to <code>127.0.0.1</code> instead of all interfaces (<code>0.0.0.0</code>).	Test URL from phone: <code>http://[PC_IP]:11434/api/tags</code> .	Set system environment variable <code>OLLAMA_HOST=0.0.0.0</code> on desktop and restart service.
Ollama Cloud 401 Unauthorized	Missing or expired Ollama Cloud API key.	Check <code>ProvidersScreen</code> connection test result for Ollama Cloud.	Obtain valid key from <code>ollama.com</code> and save in Providers → Ollama Cloud.
Custom Provider 404 / 405 on Discovery	Remote OpenAI gateway (vLLM, LM Studio) does not implement <code>GET /models</code> .	Inspect HTTP response code in Logcat connection test.	HERMAX supports Manual Model Entry : type model ID directly in Providers screen.
SSE Streaming Interrupted	Aggressive mobile battery optimization killing background socket.	Check if disconnection coincides with screen locking or switching apps.	Set HERMAX battery usage to "Unrestricted" in Android App Settings.
Build Fails with KSP Version Mismatch	Kotlin compiler version upgraded without matching KSP compiler plugin.	Inspect <code>libs.versions.toml</code> for <code>kotlin</code> and <code>ksp</code> version alignment.	Ensure <code>kotlin = "2.1.0"</code> pairs exactly with <code>ksp = "2.1.0-1.0.29"</code> .

13. Technical Glossary & Potential Extensions

13.1 Technical Glossary

- **SSE (Server-Sent Events):** A unidirectional HTTP streaming protocol where the server pushes UTF-8 formatted text chunks over a persistent TCP connection using `text/event-stream`.
- **LLM (Large Language Model):** Deep neural network models (e.g. Claude 3.5 Sonnet, GPT-4o, Llama 3) generating text tokens sequentially.
- **Android Keystore:** A hardware-backed system service on Android devices that generates and stores cryptographic keys inside secure hardware (TEE/StrongBox) to prevent key extraction.
- **GFM (GitHub Flavored Markdown):** Standard Markdown extension specification adding support for tables, task checklists, and strikethrough styling.
- **vLLM / LM Studio:** Popular open-source local inference engines that host AI models locally while exposing OpenAI-compatible `/v1/chat/completions` endpoints.

13.2 Potential Future Extensions (Not Currently Implemented)

- *On-Device NPU Local Inference:* Integration with Google MediaPipe or ONNX Runtime to execute lightweight 1B-3B quantized models directly on the mobile device processor.
- *Multi-Modal Audio Streaming:* Support for bi-directional WebRTC audio streaming to engage in real-time voice conversations.
- *Encrypted Conversation Export:* Exporting conversation databases into password-protected AES-256 backup archives.